

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **1**
Topic : **1**

Introduction to NoSQL Concepts

Objectives

This session will give the knowledge about

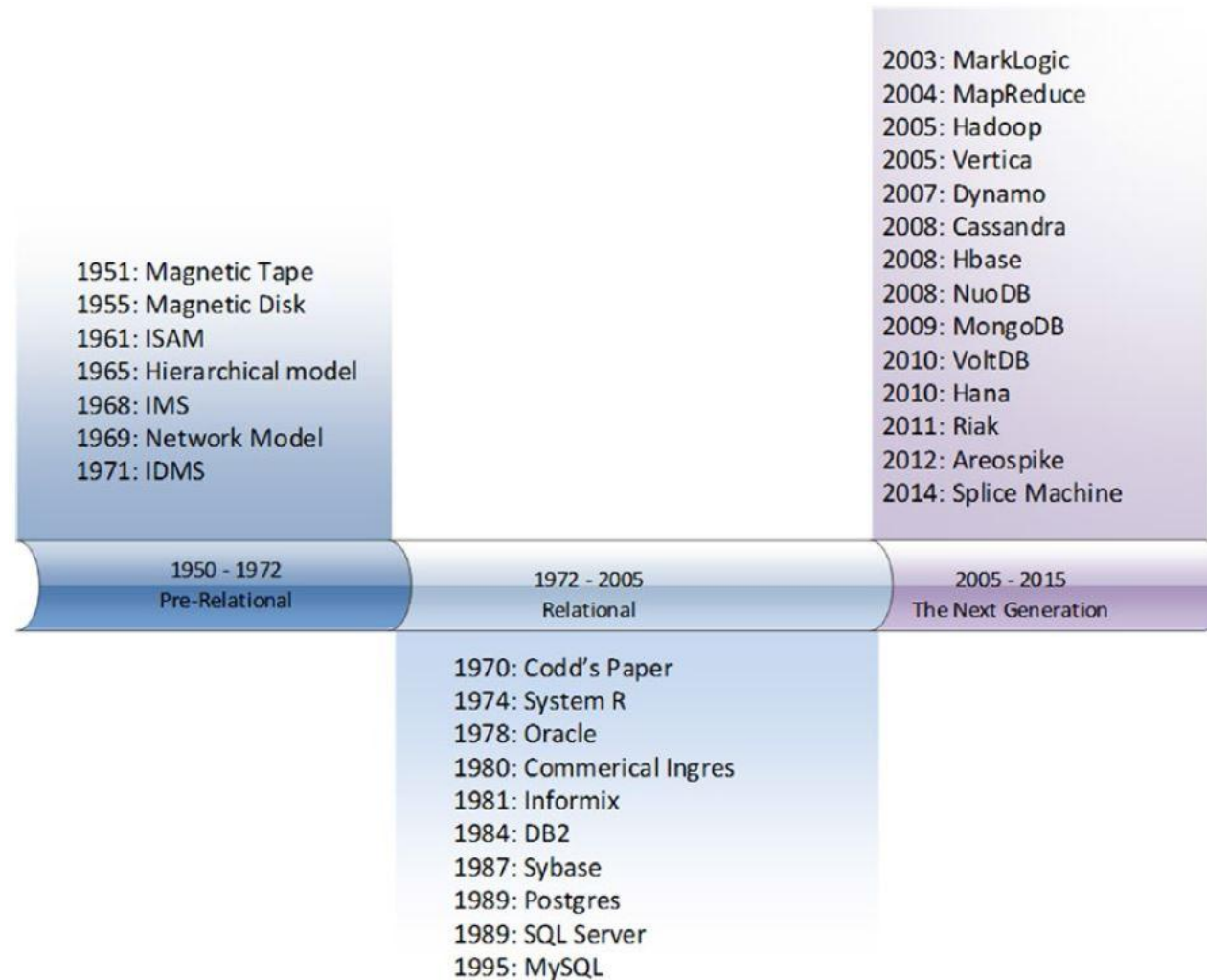
- Database revolutions
- First generation Database
- Second generation Database
- Third generation Database
- What is NoSQL?
- Comparison between SQL and NoSQL?

What is Database?

A **database** is a collection of data, typically describing the activities of one or more related entities and attributes.

A **database management system**, or DBMS, is software designed to assist in **maintaining and utilizing large collections of data**, and the need for such systems, as well as their use, is growing rapidly.

Evolution of Database



First Database Revolution

The emergence of electronic computers following the Second World War represented the **first revolution** in databases.

Early “databases” used **paper tape initially and eventually magnetic tape** to store data sequentially.

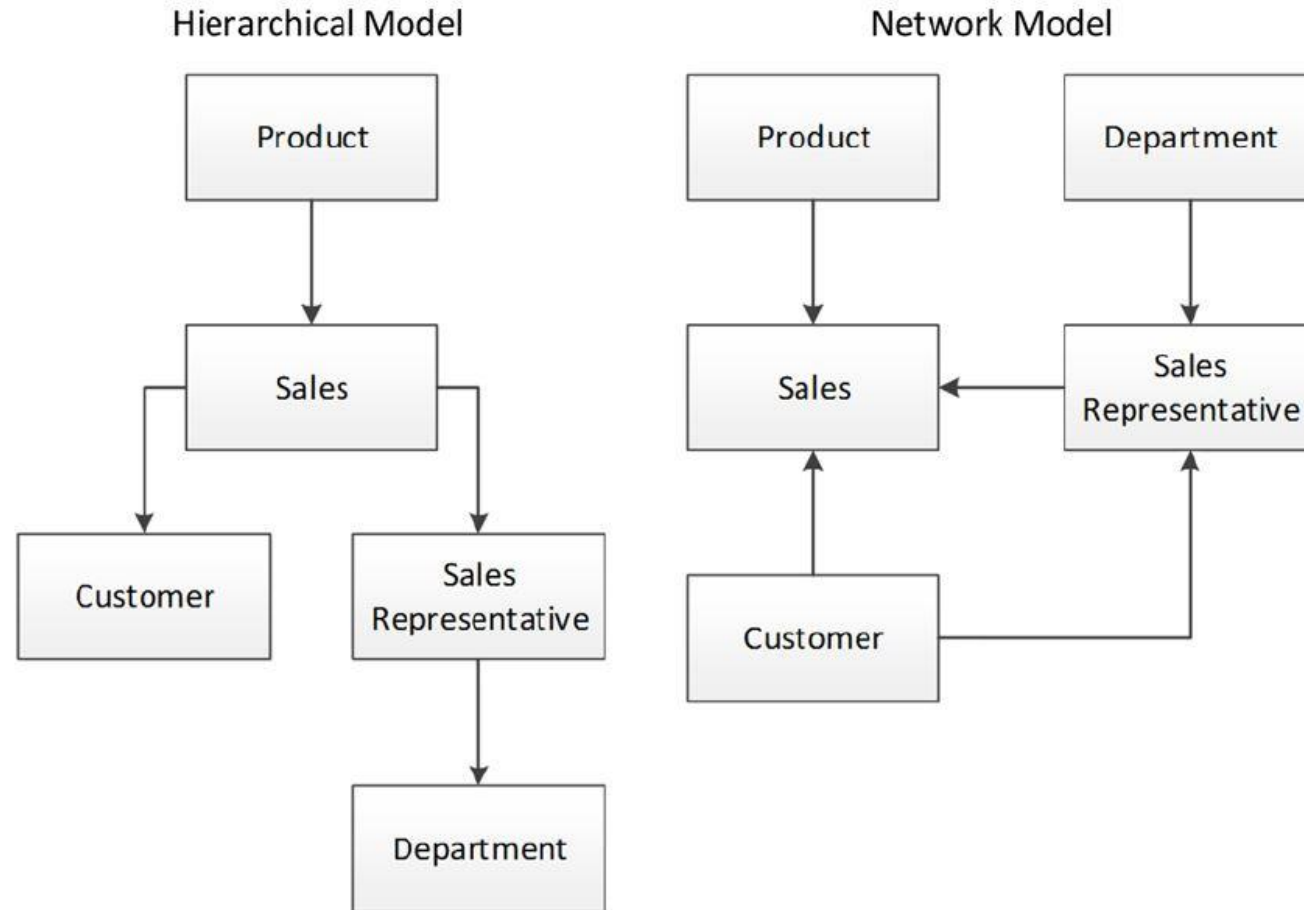
- 1955: spinning magnetic disk - **direct high-speed access** to individual records
- 1961: **ISAM (Index Sequential Access Method)** made **fast record-oriented access** feasible and consequently **leads to OLTP (On-line Transaction Processing)** computer systems.

First Database Revolution

By the early 1970s, two major models of DBMS were competing for dominance.

- The network model was formalized by the CODASYL standard and implemented in databases such as IDMS.
- The hierarchical model provided a somewhat simpler approach found in IBM's IMS (Information Management System).

Hierarchical vs Network model



Second Database Revolution

In the late 1960s, Codd who is working at an IBM laboratory, found the following drawbacks in First generation DBMS:

- Existing databases were **too hard to use**.
- Existing databases **lacked a theoretical foundation**.
- Existing databases **mixed logical and physical implementations**.

To overcome all these, he published a core ideas that **defined the relational database model that became the most significant model** for database systems for a generation

New in Second generation

Key concepts of the relational model include:

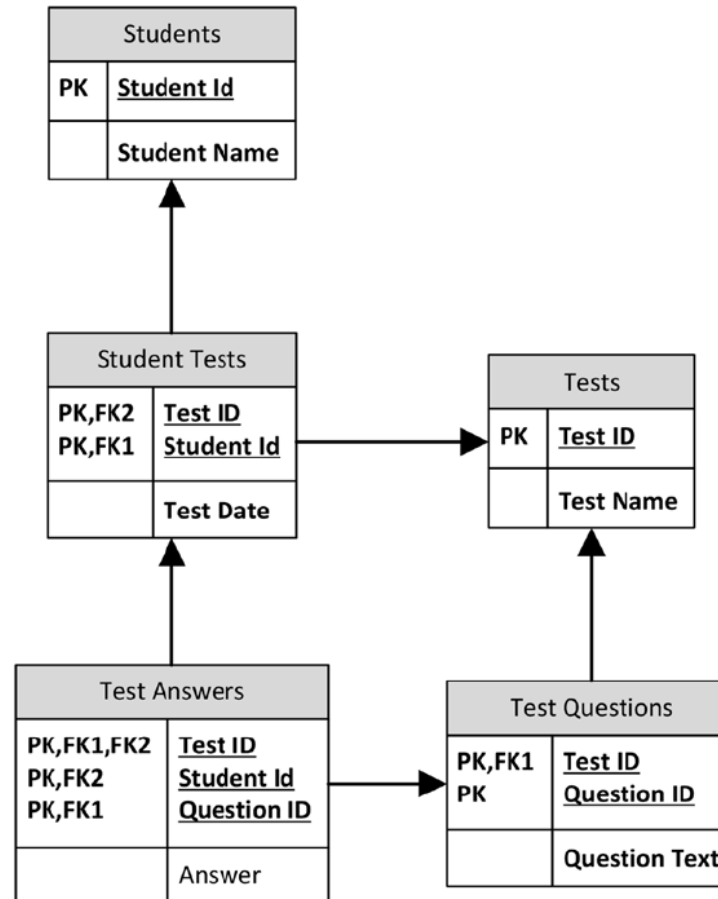
- Attributes
- Tuples
- Relation
- Constraints
- Operations
- Normal forms

Benefits in Second generation

Un-normalized data

Test scores	
Student Name	
Test Name	
Test Date	
Answer 1	
Answer 2	
Answer 3	
Answer 4	
Answer 5	
Answer 6	
Answer N	

Normalized data



Transaction Models

Jim Gray defined the most widely accepted transaction model in the late 1970s. This soon became popularized as ACID transactions

- **Atomic:** The transaction is indivisible - either all the statements in the transaction are applied to the database or none are.
- **Consistent:** The database remains in a consistent state before and after transaction execution.
- **Isolated:** While multiple transactions can be executed by one or more users simultaneously, one transaction should not see the effects of other in-progress transactions.
- **Durable:** Once a transaction is saved to the database, its changes are expected to persist even if there is a failure of operating system or hardware.

SQL

In 1970, Edgar Codd, at IBM's San Jose Research Laboratory, proposed a new data representation framework called the **relational data model**.

In the 1980s, The **SQL query language for relational databases**, developed as part of IBM's **System R project**, is now the standard query language.

SQL was standardized in the late 1980s, and the current standard, SQL-92, was adopted by the **American National Standards Institute (ANSI) and International Standards Organization (ISO)**.

During the succeeding decades many new database systems were introduced. These include **Sybase, Microsoft SQL Server, Informix, MySQL, and DB2**.

Third Database Revolution

By 2005, Google was by far the biggest website in the world.

When Google began, the relational database was already well established, **but it was inadequate to deal with the volumes and velocity of the data** confronting Google.

- In 2003, Google revealed details of the **distributed file system GFS**
- In 2004 it revealed details of the distributed parallel processing algorithm **“MapReduce”**
- In 2006, Google revealed details about its **BigTable distributed structured database.**

Drawbacks in Second Database Revolution

- Even the most expensive commercial RDBMS such as Oracle **could not provide sufficient scalability** to meet the demands of large web sites.
- To overcome this major issue, **distributed databases** has been introduced.
- **“Sharding”** involves partitioning the data across multiple databases based on **a key attribute**, such as the customer identifier.
- Sharding at sites like Facebook has allowed a MySQL-based system to scale up to massive levels, but the downsides of doing this are immense. **Many relational operations and database-level ACID transactions are lost.**

Cloud Computing

- Between 2006 and 2008, Amazon rolled out **Elastic Compute Cloud (EC2)**.
- EC2 made available **virtual machine images** hosted on Amazon's hardware **infrastructure and accessible via the Internet**.
- Amazon added other services such as **storage (S3, EBS)**, **Virtual Private Cloud (VPC)**, **a MapReduce service (EMR)**, and so on.
- The entire platform was known as **Amazon Web Services (AWS)** and was the first practical implementation of an **Infrastructure as a Service (IaaS) cloud**.
- AWS became the inspiration for cloud computing offerings from Google, Microsoft, and others.

Document Databases

- The impedance mismatch between object-oriented and relational models, leads to **Object relational mapping systems**.
- This was enabled by the programming style known as **AJAX (Asynchronous JavaScript and XML)**, in which JavaScript within the **browser communicates directly with a backend** by transferring XML messages.
- XML was soon superseded by **JavaScript Object Notation (JSON)**, which is **a self-describing format similar to XML** but is **more compact and tightly integrated** into the JavaScript language.
- The databases which supports JSON to directly create and access the database and eliminates the role of relational middle man is known as “**Document Databases**”.
- **CouchBase and MongoDB** are two popular JSON-oriented databases, though virtually all nonrelational databases

NewSQL

In 2007, Michael Stonebraker and his team proposed a number of variants on the existing RDBMS design.

- H-Store described a pure in-memory distributed database
- C-Store specified a design for a columnar database.

Both these designs were extremely influential in the years to come and are the first examples of what came to be known as NewSQL database systems

NewSQL databases that retain key characteristics of the RDBMS but that diverge from the common architecture exhibited by traditional systems such as Oracle and SQL Server.

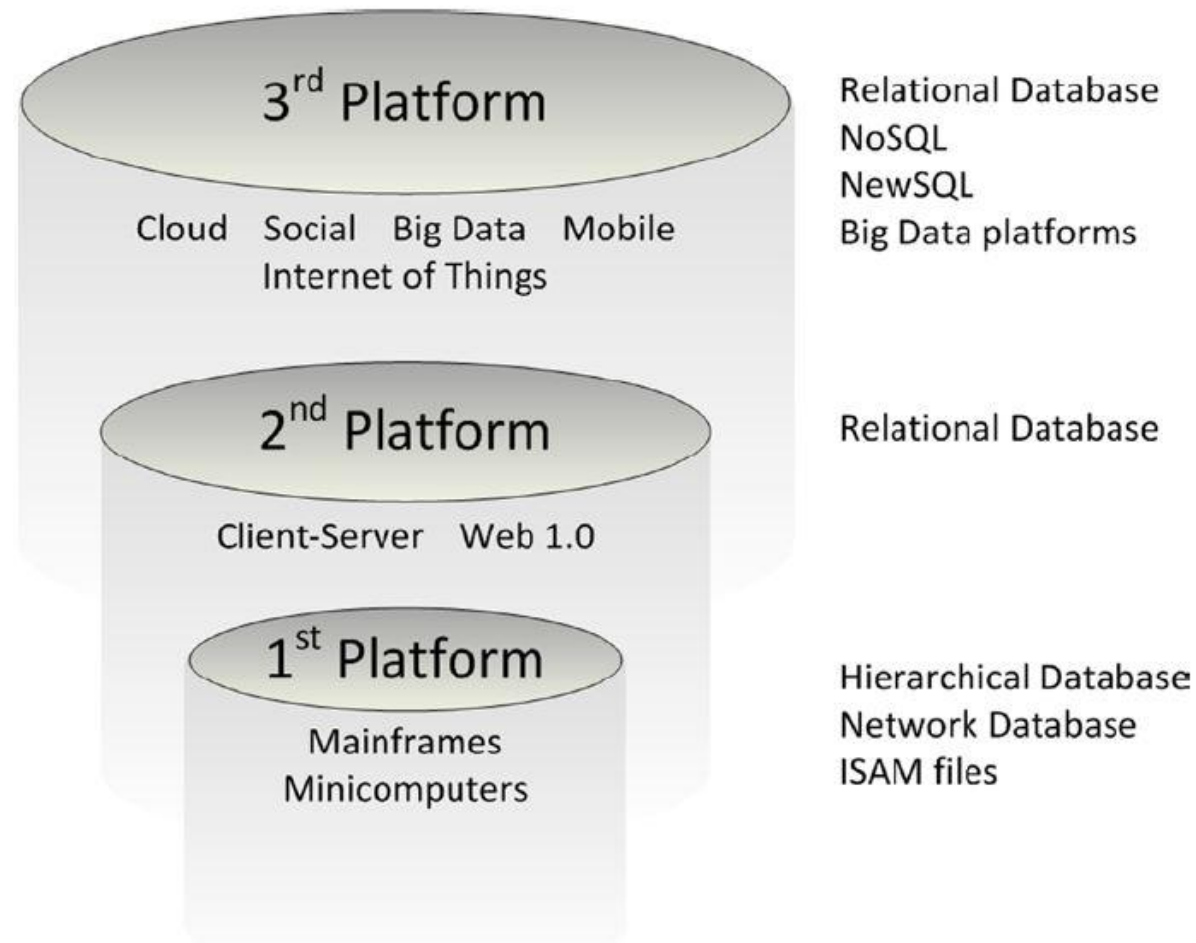
The Nonrelational Explosion

At the conclusion, dozens of new database systems like such as **MongoDB**, **Cassandra**, and **HBase** emerged due to the drawbacks of relational databases.

These new breeds of database systems lacked a common name “Distributed **Non-Relational Database Management System**” (DNRDBMS).

However, in late 2009, the term **NoSQL** quickly caught on as shorthand for any database system that broke with the traditional SQL database.

The Database technologies



What is NoSQL?

- NoSQL database, also called **Not Only SQL**, is an **approach to data management and database design** that's **useful for very large sets of distributed data**.
- NoSQL is **not a relational database**
- A relational database model may not be the best solution for all situations.
- The easiest way to understand NoSQL, is that of **a database which does not adhering to the traditional relational database management system (RDMS) structure**.

What is NoSQL?

- The most popular NoSQL database is **Apache Cassandra**.
- Cassandra, which was once **Facebook's proprietary database**, was released as open source in **2008**.
- Other NoSQL implementations include **SimpleDB, Google BigTable, Apache Hadoop, MapReduce, MemcacheDB, and Voldemort**.
- Companies that use NoSQL include **NetFlix, LinkedIn and Twitter**.

Why we should use NoSQL?

There are several reasons why people consider using a NoSQL database.

- Application development productivity.
- Large data.
- Analytics.
- Scalability.
- Massive write performance.
- Fast key-value access.
- Flexible data model and flexible datatypes.
- Schema migration.
- Write availability.
- Easier maintainability, administration and operations.
- Generally available parallel computing.
- Programmer ease of use.
- Distributed systems and cloud computing support.

SQL vs NoSQL

SQL	NoSQL
Relational Databases (RDBMS)	Non-relational or distributed database
Table based databases	Document based, key-value pairs, graph databases or wide-column stores
Have predefined schema	Have dynamic schema for unstructured data
Vertically scalable	Horizontally scalable
Uses SQL (structured query language)	Uses UnQL (Unstructured Query Language). The syntax of using UnQL varies from database to database

SQL vs NoSQL

SQL	NoSQL
MySql, Oracle, Sqlite, Postgres and MS-SQL	MongoDB, BigTable, Redis, RavenDb, Cassandra, Hbase, Neo4j and CouchDB
Good fit for the complex query	Not good fit for complex queries (NoSQL don't have standard interfaces)
Not best fit for hierarchical data storage	Fits better for the hierarchical data storage
Scalability is managed by increasing the CPU, RAM, SSD, etc	Scalability is managed by adding few more servers easily in your NoSQL database
Best fit for heavy duty transactional type applications	Not fit for heavy transactional applications

SQL vs NoSQL

SQL	NoSQL
Excellent support are available for all SQL database	Still have to rely on community support
Emphasizes on ACID properties (Atomicity, Consistency, Isolation and Durability)	Follows the Brewers CAP theorem (Consistency, Availability and Partition tolerance)
Classified as either open-source or close-sourced	Classified on the basis of way of storing data as graph databases, key-value store databases, document store databases, column store database and XML databases.

NewSQL

- NewSQL is a **modern relational database system** that bridges the gap between SQL and NoSQL.
- NewSQL databases **aim to scale and stay consistent**.
- NewSQL is a unique database system that combines **ACID compliance with horizontal scaling**.
- The database system strives to keep the best of both worlds.
- OLTP-based transactions and the high performance of NoSQL combine in a single solution.

NewSQL - Advantages

- The main features of NewSQL databases are:
 - **In-memory storage** and data processing supply fast query results.
 - **Partitioning** scales the database into units.
 - Queries execute on many shards and combine into a single result.
 - **ACID** properties preserve the features of RDBMS.
 - **Secondary indexing** results in faster query processing and information retrieval.
 - **High availability** due to the database replication mechanism.

NewSQL: Comparision

Feature	SQL	NoSQL	NewSQL
Schema	Relational (table)	Schema-free	Both
SQL	Yes	Depends on the system	Yes, with enhanced features
ACID	Yes	No (BASE)	Yes
OLTP	Partial support	Not supported	Full support
Scaling	Vertical	Horizontal	Horizontal
Distributed	No	Yes	Yes
High availability	Custom	Auto	Built-in
Queries	Low complexity queries	High complexity queries	Both

NewSQL Examples

Some well known NewSQL Databases

- Apache Trafodion.
- Clustrix.
- CockroachDB.
- Couchbase.
- Google Spanner.
- NuoDB.
- Pivotal GemFire XD.

Summary

This session will give the knowledge about

- Database revolutions
- First, Second and Third generation Database
- What is NoSQL?
- Comparison between SQL and NoSQL?
- What is NewSQL?
- Comparison between SQL, NoSQL and NewSQL?